

Plano de Implementação — Modelagem Preditiva

Machine Learning com scikit-learn, XGBoost e SHAP · projeções 2024–2025

Plano de Implementação — Modelagem Preditiva com Machine Learning

Projeto: Data Base Analysis — Nascidos Vivos Brasileiros
(SINASC/DataSUS 2019–2023)

Status: Implementado ☒ — Fase 1 (modelagem) e Fase 2
(dashboard) concluídas. Este documento é o registro do plano, do
método e dos artefatos finais.

Ambiente detectado: Python 3.12.10 ☒ · Node v24 ☒ · R ☐ (não
instalado)

Base: 5.510 registros limpos · 9 variáveis originais + derivadas

0. Premissas e decisões de arquitetura

Decisão	Escolha	Motivo
Linguagem de execução	**Python 3.12**	Única stack de análise disponível na máquina; R (Rmd) vira documentação de referência
Libs principais	`scikit-learn`, `xgboost`, `shap`, `matplotlib`, `seaborn`	Padrão de mercado para dados tabulares; XGBoost recomendado na análise anterior
Modelo principal	**XGBoost (Gradient Boosting)**	Melhor custo-benefício performance × interpretabilidade em dados tabulares pequenos
Baselines	`DummyClassifier` + **Regressão Logística**	Define o "piso" de qualidade e o modelo interpretável
Alvos priorizados	1. `BAIXO\_PESO` · 2. `PREMATURO` · 3. `PESO\_GRAMAS` (regressão)	Os 3 problemas de saúde pública mais relevantes do dashboard
Métricas	ROC-AUC, PR-AUC, F1, sensibilidade, especificidade	Classes desbalanceadas → acurácia é enganosa
Anti-vazamento	`PESO\_GRAMAS` e `SEMANAS\_GESTACAO` excluídos conforme o alvo	Essas variáveis **definem** os alvos derivados (vazamento = modelo trivial)

1. Definição dos problemas de ML

1.1 Tarefas priorizadas

#	Tarefa	Tipo	Alvo (y)	% positivos	Features (X)
1	Prever baixo peso ao nascer	Classificação binária	<code>`BAIXO_PESO` (< 2500g)</code>	~1,8%	<code>`IDADE_MAE`</code> , <code>`CONSULTAS_PRENATAL`</code> , <code>`SEMANAS_GESTACAO`</code> , <code>`SEXO`</code> , <code>`REGIAO`/`UF`</code> , <code>`ANO`</code> , <code>`TIPO_PARTO`</code>
2	Prever prematuridade	Classificação binária	<code>`PREMATURO` (< 37 sem)</code>	~10%	<code>`IDADE_MAE`</code> , <code>`CONSULTAS_PRENATAL`</code> , <code>`SEXO`</code> , <code>`REGIAO`/`UF`</code> , <code>`ANO`</code> , <code>`TIPO_PARTO`</code>
3	Estimar peso ao nascer	Regressão	<code>`PESO_GRAMAS`</code>		– idem tarefa 1 (sem as variáveis que definem o alvo quando aplicável)
4	Prever tipo de parto	Classificação binária	<code>`TIPO_PARTO`</code>	~56,6% (cesárea)	**Opcional** – objetivo mais de associação que de predição
5	Perfil de UFs	Clusterização	grupos de UF		– **Opcional** – K-Means sobre indicadores regionais

1.2 Regras anti-vazamento (crítico)

- `BAIXO_PESO` é definido por `PESO_GRAMAS` → `PESO_GRAMAS` nunca entra como feature na tarefa 1.
- `PREMATURO` é definido por `SEMANAS_GESTACAO` → `SEMANAS_GESTACAO` não entra na tarefa 2 (prever prematuridade a partir da própria idade gestacional é circular).
- `APGAR5` só é medido após o nascimento → se o objetivo é predição antecipada (antes do parto), excluir; se é análise de associação, pode entrar. Decisão a confirmar.
- Toda a lógica deve ficar documentada em código para reuso acadêmico (alinhada ao espírito do Rmd).

2. Etapa 1 — Setup do ambiente (10 min)

```
# Criar ambiente virtual e instalar dependências
python -m venv .venv
source .venv/bin/activate # Windows (bash):
.venv/Scripts/activate
pip install pandas numpy scikit-learn xgboost shap
matplotlib seaborn jupyter
```

Estrutura de pastas a criar:

```
ml/
├─ pipeline.py # motor da análise (limpeza, modelagem,
avaliação, SHAP)
├─ nascimentos_ml.ipynb # notebook principal (toda a
análise)
├─ requirements.txt # dependências com versões fixadas
├─ resultados/
├─ metricas_modelos.csv # comparação de todos os modelos
(por tarefa)
├─ metricas_regressao.csv # métricas da regressão de
PESO_GRAMAS
├─ curvas_<tarefa>.png # curvas ROC + Precision-Recall
(out-of-fold)
├─ confusao_<tarefa>.png # matriz de confusão do melhor
modelo (OOF)
├─ shap_summary_<tarefa>.png # SHAP global (importância
de features)
└─ scatter_peso_gramas.png # previsto × real (regressão)
```

3. Etapa 2 — Preparação de dados (1–2 h)

Carregar e replicar a limpeza documentada no `data-base-analysis.Rmd`:

- Padronizar SEXO, converter PESO_GRAMAS, filtrar IDADE_MAE 10–55, validar UF, deduplicar.

Criar features derivadas (mesmas regras do Rmd): BAIXO_PESO, PREMATURO, FAIXA_IDADE_MAE, TIPO_PARTO_LABEL.

Enriquecer: JOIN com uf_referencia.csv → REGIAO (manter UF como feature opcional com target encoding ou excluir para evitar 27 categorias esparsas).

Feature engineering leve: IDADE_MAE² (relação não-linear com risco), interação IDADE_MAE × CONSULTAS_PRENATAL (opcional).

Codificação: one-hot para SEXO/REGIAO; StandardScaler apenas para Regressão Logística (árvores não precisam).

Split: train_test_split estratificado 70/30, mantendo a proporção da classe rara.

Tratar desbalanceamento: class_weight='balanced' no Logistic Regression / scale_pos_weight no XGBoost; SMOTE só dentro do K-Fold (nunca antes do split, para não vazar dados sintéticos para o treino/validação).

4. Etapa 3 — Modelos e validação (2–4 h)

4.1 Pipeline de avaliação (igual para todos)

```
StratifiedKFold(cv=5, shuffle=True, random_state=42)
```

→ por dobra: treinar no treino, medir no val
→ reportar média ± desvio de: ROC-AUC, PR-AUC, F1, sensibilidade, especificidade

4.2 Modelos em ordem de construção

Ordem	Modelo	Papel	Configuração inicial
1	<code>DummyClassifier</code>	(majority)	Piso
2	<code>LogisticRegression</code>	Baseline interpretável	<code>class_weight='balanced'</code> , <code>max_iter=1000</code> , escalado
3	<code>RandomForestClassifier</code>	Sanity check robusto	300 árvores, <code>class_weight='balanced'</code>
4	<code>XGBClassifier</code>	Modelo principal	<code>learning_rate=0.05</code> , <code>max_depth=3-5</code> , <code>scale_pos_weight</code> , <code>eval_metric='aucpr'</code> , early stopping

4.3 Tuning (somente no XGBoost)

RandomizedSearchCV com 5-fold estratificado:

- `max_depth`: [3, 4, 5, 6]
- `learning_rate`: [0.01, 0.05, 0.1]
- `n_estimators`: 200–500 com early stopping
- `subsample/colsample_bytree`: [0.7, 0.9, 1.0]
- `scale_pos_weight`: calculado de neg/pos e variantes {0.5×, 1×, 2×}

4.4 Otimização do limiar

- Após escolher o melhor modelo, otimizar o threshold no conjunto de validação (Youden's J = maximizar sensibilidade + especificidade – 1, ou maximizar F1).
- Mostrar o trade-off sensibilidade × especificidade no relatório (decisão de saúde pública).

5. Etapa 4 — Interpretação com SHAP (1 h)

shap.TreeExplainer sobre o XGBoost final.

Importância global: SHAP summary/beeswarm plot → quais fatores mais elevam/reduzem o risco de baixo peso e prematuridade.

Análise de dependência (SHAP dependence plots):

- IDADE_MAE × risco → confirmar a forma em U (adolescentes e mães 35+).
- CONSULTAS_PRENATAL × risco → efeito protetivo do pré-natal.

Caso prático (explicação local): 2–3 exemplos de predição com force plot, para mostrar por que o modelo deu alto/alto risco — essencial no contexto de saúde pública.

Alternativa de interpretabilidade pura (acadêmica): treinar uma árvore de decisão única com max_depth=3 apenas para leitura visual das regras (ex.: "mãe < 18 anos E pré-natal < 4 consultas → maior risco"). Não é o modelo final, é ferramenta de explicação.

6. Etapa 5 — Entrega (1 h)

metricas_modelos.csv: comparativo final (tabela no relatório).

RELATORIO.md: resumo executivo com:

- Tabela de comparação de modelos (AUC, PR-AUC, F1).

-
- Ranking de importância de features (SHAP).
 - Insights de saúde pública traduzidos do dashboard (ex.: região Centro-Oeste com mais prematuridade aparece no modelo?).
 - Decisões de limiar recomendadas.
- Gráficos salvos em `ml/resultados/`.
-

7. Fase 2 — Integração no dashboard React (implementado)

Objetivo atingido: exibir os resultados de ML e a calculadora sem rodar modelo no browser (cálculo 100% client-side). Como a Regressão Logística venceu o XGBoost (sinal majoritariamente linear), basta exportar ~11 coeficientes + scaler.

```
| Artefato | Papel |
| `ml/export_frontend_data.py` | Exporta o modelo LR
(scaler + coeficientes + limiar de Youden), SHAP, ROC e
métricas → `src/data/ml.ts` (verifica `sigmoid` manual ==
sklearn) |
| `src/data/ml.ts` | Dados tipados do modelo (1.099
linhas) |
| `src/components/Predictive/predictModel.ts` | Matemática
client-side: `buildVector()`, `predict()`,
`impactPotential()` – funções puras e testáveis |
| `src/components/Predictive/Predictive.tsx` | Seção
**"Modelagem"**: calculadora de risco (sliders + chips +
gauge + badge + fatores + indicadores de impacto ± p.p.) e
painel de resultados (SHAP, comparativo 5-fold CV, curvas
ROC, abas baixo peso/prematuridade) |
| `src/components/Predictive/Forecast.tsx` | Bloco
**Projeções 2024–2025** (detalhado na seção 8) |
| `src/components/Predictive/Predictive.css` | Estilos do
bloco (cyber, dark/light, responsivo) |
| `src/test/predictModel.test.ts` ·
`src/test/forecast.test.ts` | Testes unitários da
matemática e da integridade dos dados exportados |
```

Validação: npm run test:run (95 testes) · npm run lint (0 erros) · npm run build — todos verdes.

8. Projeções 2024–2025 (método top-down, implementado)

Especificação completa (fórmulas, números reais e estrutura de dados): ml/PROJECÃO_TECN_DESIGN.md

Implementação: ml/export_forecast.py → src/data/forecast.ts + bloco no front.

8.1 Por que top-down (e não ARIMA/Prophet ou OLS por série)

Com 5 pontos anuais (2019–2023), modelos de série temporal não são defensáveis. A análise real dos dados mostrou que as participações (%) são mais estáveis e informativas que os totais: a tendência de declínio do Norte só foi significativa na share ($p = 0,044$ vs $p = 0,110$ nos totais brutos). Projetar cada série isoladamente é incoerente (os grupos não somam o total); o top-down garante a consistência por construção.

8.2 Método em 3 camadas

Total — OLS $\hat{T} = a + b \cdot t$ com intervalo de predição de 95% (inclui o erro do processo). Resultado 2025: 1.100,4 [941,5–1.259,3].

Participações — share de cada grupo ancorada no último valor observado (2023) + drift da inclinação OLS da série de shares, com clamp e normalização (soma = 100%). A ancoragem evita os saltos implausíveis do OLS puro (caso documentado: Sexo Feminino projetava 46,75% vs 50,25% ancorado).

Reconciliação — $\hat{y}_i = \hat{T} \times \text{share}_i \rightarrow$ cada dimensão soma exatamente o total projetado (verificado nos testes).

8.3 Incerteza (método delta de 1ª ordem)

$\text{Var}(\hat{y}_i) \approx (\text{share}/100)^2 \cdot \text{Var}(T) + (T/100)^2 \cdot \text{Var}(\text{share})$ — banda indicativa (shares correlacionadas por construção). A incerteza cresce com o horizonte, o que justifica limitar a projeção a 2 anos.

8.4 Validação holdout (treino 2019–2022 → teste 2023)

- Total: previsão 1.088 vs real 1.110 → erro -2,0%.
- Shares (MAE em p.p.): região 1,9 · faixa etária 1,85 · sexo 8,07 (série em "V" — variação real dos dados, reportada honestamente).

8.5 Destaques (únicos sinais de tendência detectáveis)

```
| Série | Drift (p.p./ano) | p | Veredito |  
| **Norte** (participação) | -0,39 | **0,044** | Tendência  
significativa |  
| **Mães 35+** (participação) | -0,75 | 0,059 | Quase  
significativa |
```

8.6 Dados exportados (src/data/forecast.ts)

- forecastData.total — série única com history + projection (lo/hi = banda 95%).
- forecastData.regiao / faixaEtaria / sexo — séries com history, projection, shareHistory, shareProjection, slopePp, pValue.

```
- `forecastData.highlights` — ids das séries com  $p < 0,10$   
e  $|\text{drift}| \geq 0,2$  p.p./ano.
```

-
- `forecastData.validation` — holdout do total + MAE das shares por dimensão.

8.7 Front

(`src/components/Predictive/Forecast.tsx`)

- Gráfico SVG: histórico sólido + projeção tracejada + banda de incerteza sombreada (95%).
 - Chips de dimensão (Total/Região/Faixa etária/Sexo) + chips de série + toggle Nascimentos ↔ Participação %.
 - Cards de destaque (Norte ↓, 35+ ↓) e card de validação holdout (erro do total + MAE).
 - Nota de honestidade: "projeção exploratória de tendência com incerteza quantificada".
-

9. Riscos e mitigações

Risco	Mitigação
Overfitting (dataset de 5.510 linhas)	Validação cruzada, árvores rasas, early stopping, regularização
Classe rara (1,8% de baixo peso)	PR-AUC como métrica principal, `class\_weight`/`scale\_pos\_weight`, otimização de threshold
Vazamento de dados	Regras da seção 1.2, sempre com check automatizado no pipeline
SMOTE vazando dados	SMOTE aplicado ****dentro**** de cada fold, nunca no dataset inteiro
R indisponível	Execução em Python; Rmd mantido como referência acadêmica
27 categorias de UF esparsas	Uso de `REGIAO` como feature principal; UF com target encoding se necessário

10. Cronograma consolidado

Etapa	Duração estimada	Entregável
1. Setup	10 min	ambiente pronto, estrutura `ml/`
2. Dados	1-2 h	notebook com limpeza + split + encoding
3. Modelos	2-4 h	4 modelos comparados, tuning, threshold
4. SHAP	1 h	gráficos + interpretação local
5. Entrega	1 h	conclusão interpretada no notebook `nascimentos\_ml.ipynb` + `ml/resultados/`
****Fase 1 total****	****~1 dia****	análise preditiva completa 📄
6. Fase 2 (dashboard)	4-8 h	seção "Modelagem" + Projeções no dashboard + testes 📄

11. Critérios de aceite

- [x] Anti-vazamento verificado em código (sem PESO_GRAMAS no alvo 1; sem SEMANAS_GESTACAO no alvo 2).
- [x] XGBoost supera a Regressão Logística em PR-AUC no hold-out — não superou: a LR venceu nas duas tarefas, virando a recomendação (comportamento previsto pelo plano: "se não superar, a LR vira a recomendação — ok também").
- [x] Métricas reportadas com média $\pm$ desvio de 5-fold, nunca só hold-out.
- [x] SHAP incluído: gráficos globais por tarefa em `ml/resultados/shap_summary_*.png` + importância exportada para o front.
- [x] Interpretação em linguagem de saúde pública — conclusão do notebook `ml/nascimentos_ml.ipynb` e da seção "Modelagem" no dashboard.
- [x] Projeções validadas por holdout e com banda de incerteza honesta (erro $-2,0\%$ no total).